

PROYECTO HAPPINESS&CO

David Blanco Martín

Nuestro proyecto Happiness Co simula ser una empresa que organiza distintos eventos culturales para las personas en Asturias. La empresa para llevar a cabo el proceso de organización y gestión del personal, recursos y demás para los eventos debe crear un programa en java(Programación) que gestione los eventos, personas, fechas, usuarios etc, que van a participar en los eventos.

También necesitamos una base de datos de MySQL(Base de datos) para gestionar la información de los participantes.

Finalmente para que el usuario pueda contactar con nosotros, apuntarse a los eventos y en resumen ser incluido en la base de datos a través del programa de java necesitamos una página web (Lenguaje de marcas).

link GitHub: <https://github.com/Davidovich97/proyectoHappinessDavidBlanco>

● BLOQUE PROGRAMACIÓN

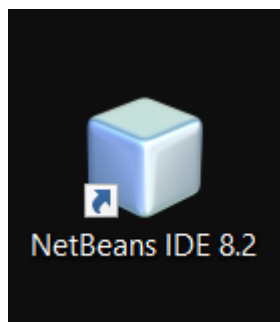
El objetivo principal del bloque de Programación en el proyecto HAPPINESS Co es desarrollar un programa para la gestión de datos en memoria utilizando la programación basada en objetos de Java. El programa debe actuar como un sistema de administración que permita gestionar en tiempo real: usuarios, eventos, galerías fotográficas y las preferencias (favoritos) de los clientes.

Objetivos del programa en java :

- Crear modelos de datos: Crear una estructura de clases sólida (Usuarios, Eventos, Galerías y Favoritos) representando las partes de la empresa con las que vamos a trabajar.
- Encapsulamiento y Calidad de Código: Garantizar que cada clase posea sus atributos privados, constructores definidos y métodos de acceso (Getters/Setters) y representación (toString) para asegurar un código limpio y profesional bien refactorizado.
- Aprender a utilizar estructuras de datos avanzadas como:
 - HashMaps: Para garantizar que no haya repeticiones de elementos clave, como el correo de los usuarios o los identificadores de eventos.

- ArrayLists: Para permitir el almacenamiento de sub-elementos, como las colecciones de galerías dentro de cada evento.
- Interfaz de Usuario: Crear un sistema de menú mediante un bucle do-while que permite al usuario realizar todas las acciones para obtener información, crear o eliminar elementos de forma intuitiva por pantalla.
- Control de errores: Programar mecanismos que impidan errores de integridad, como avisar si un usuario ya existe antes de añadirlo o verificar que un ID de evento sea correcto antes de intentar borrarlo o añadirle una galería.
- Relaciones: Simular la interconexión entre las distintas partes del sistema, permitiendo que un usuario pueda marcar eventos específicos como "favoritos" relacionando su correo con el ID del evento.

Para desarrollar el apartado de programación comencé usando Visual Studio Code pero me pasé a usar NetBeansIDE para crear las clases y comprobar su funcionamiento.



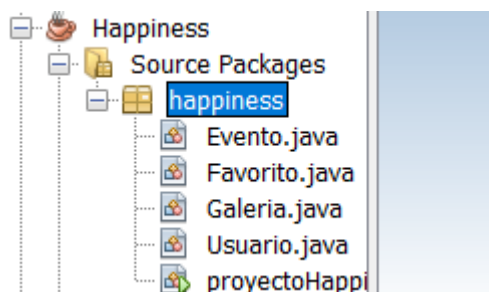
Primero desarrollamos las clases que sirven para crear los objetos, estas son: evento, usuario, galería y favorito etc. Me he documentado en las siguientes páginas para aprender sobre HashMaps y ArrayList:

<https://docs.oracle.com/javase/8/docs/api/java/util/HashMap.html>

https://www.w3schools.com/java/java_arraylist.asp

<https://www.baeldung.com/java-collections>

Primero comenzamos desarrollando las clases y luego la clase main:



Cada clase de objeto nos pide un método para crear el objeto con sus variables, getters, setters y método ToString. Nos queda como ejemplo Evento.java así:

```
public class Evento {
    private int id;
    private LocalDate fecha;
    private String titulo;
    private String ubicacion;
    private String descripcion;
    private ArrayList<Galeria> galerias;

    public Evento(int id, LocalDate fecha, String titulo, String ubicacion, String descripcion) {
        this.id = id;
        this.fecha = fecha;
        this.titulo = titulo;
        this.ubicacion = ubicacion;
        this.descripcion = descripcion;
        this.galerias = new ArrayList<>();
    }

    // Getters y Setters
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public LocalDate getFecha() { return fecha; }
    public void setFecha(LocalDate fecha) { this.fecha = fecha; }
    public String getTitulo() { return titulo; }
    public void setTitulo(String titulo) { this.titulo = titulo; }
    public String getUbicacion() { return ubicacion; }
    public void setUbicacion(String ubicacion) { this.ubicacion = ubicacion; }
    public String getDescripcion() { return descripcion; }
    public void setDescripcion(String descripcion) { this.descripcion = descripcion; }
    public ArrayList<Galeria> getGalerias() { return galerias; }
    public void setGalerias(ArrayList<Galeria> galeriaList) { this.galerias = galerias; }

    @Override
    public String toString() {
        return "id" + id + ", fecha" + fecha + ", titulo" + titulo + ", ubicacion" + ubicacion;
    }
}
```

Hacemos lo mismo con las clases de Usuario:

```
package Happiness;
public class Usuario {
    private String nombre;
    private String email;
    private String password;

    public Usuario(String nombre, String email, String password) {
        this.nombre = nombre;
        this.email = email;
        this.password = password;
    }

    // Getters
    public String getNombre()
    { return nombre; }
    public void setNombre(String nombre)
    { this.nombre = nombre; }
    public String getEmail()
    { return email; }

    // Setters
    public void setEmail(String email)
    { this.email = email; }
    public String getPassword()
    { return password; }
    public void setPassword(String password)
    { this.password = password; }
}
```

Galerías:

```
package Happiness;
public class Galeria {
    private int id;
    private String titulo;
    private int idEvento;

    public Galeria(int id, String titulo, int idEvento) {
        this.id = id;
        this.titulo = titulo;
        this.idEvento = idEvento;
    }

    // Getters
    public int getId() { return id; }
    public String getTitulo() { return titulo; }
    public int getIdEvento() { return idEvento; }

    // Setters
    public void setId(int id) { this.id = id; }
    public void setTitulo(String titulo) { this.titulo = titulo; }
    public void setIdEvento(int idEvento) { this.idEvento = idEvento; }

    @Override
    public String toString() {
        return "id" + id + " titulo" + titulo + " idEvento" + idEvento;
    }
}
```

Favoritos:

```
package Happiness;
public class Favorito {
    private String correoUsuario;
    private int idEvento;

    public Favorito(String correoUsuario, int idEvento) {
        this.correoUsuario = correoUsuario;
        this.idEvento = idEvento;
    }

    // Getters
    public String getCorreoUsuario() {
        return correoUsuario;
    }
    public int getIdEvento() {
        return idEvento;
    }

    // Setters
    public void setCorreoUsuario(String correoUsuario) {
        this.correoUsuario = correoUsuario;
    }

    public void setIdEvento(int idEvento) {
        this.idEvento = idEvento;
    }

    @Override
    public String toString() {
        return correoUsuario + idEvento;
    }
}
```

Una vez creados las clases de usuario, evento, favorito y galería. Vamos a crear la clase main en la que crearemos el menú y trabajaremos con las clases y sus métodos ya creados. Para ello nos piden que el programa debe permitir al usuario a través del menú poder crear o eliminar usuarios, eventos, galerías y favoritos. Creamos una variable para la respuesta del menú y hacemos un bucle do-while:

```
// Bucle para escoger opcion del menu
do {System.out.println("\n MENU HAPPINESS&Co\n");
    System.out.println("1. Añadir usuario");
    System.out.println("2. Eliminar usuario");
    System.out.println("3. Añadir evento");
    System.out.println("4. Eliminar evento");
    System.out.println("5. Añadir galería a evento");
    System.out.println("6. Eliminar galería de evento");
    System.out.println("7. Añadir favorito");
    System.out.println("8. Eliminar favorito");
    System.out.println("9. Salir");
    System.out.print("Seleccione una opción: ");
    opcionMenu=teclado.nextInt();
    teclado.nextLine();
```

Creamos al principio de la clase los hashmaps y el arraylist donde vamos a guardar los datos de eventos usuarios e imágenes junto con los contadores para actualizar los ID

```
HashMap<String, Usuario> mapaUsuarios = new HashMap<>();
HashMap<Integer, Evento> mapaEventos = new HashMap<>();
ArrayList<Favorito> listaFavoritos = new ArrayList<>();

// Contadores necesarios para los ID
int contadorIdsEventos=0;
int contadorIdsGalerias=0;
int idEvento;
```

Importamos el siguiente código para usar la fecha, usar el Scanner y diferentes funciones:

```

package Happiness;
import java.time.LocalDate;
import java.util.*;

```

La primera opción de nuestro switch es **crear usuario**, en este apartado asignamos los valores a las variables del usuario por teclado:

```

//Añadir Usuario
case 1:

    System.out.print("Introduce Nombre: ");
    String nombreNuevo = teclado.nextLine();
    System.out.print("Introduce Email: ");
    String emailNuevo = teclado.nextLine();
    System.out.print("Introduce Password: ");
    String passNuevo = teclado.nextLine();

```

Comprobamos que no existe ya el usuario en nuestro HashMap y si no existe lo crea, si ya existe nos da un mensaje de error:

```

//Verificamos si ya tenemos el usuario creado teniendo en cuenta el email que es la Clave
if (mapaUsuarios.containsKey(emailNuevo)) {
    System.out.println("El usuario ya existe");

//Si no existe lo guardamos con .put en el HashMap de usuarios guardando
//el email nuevo como clave y el usuario nuevo como el valor
} else {
    mapaUsuarios.put(emailNuevo, new Usuario(nombreNuevo, emailNuevo, passNuevo));
    System.out.println("Usuario creado correctamente");
}
break:

```

Case 2:

En el case 2 queremos **eliminar usuario**, pedimos por teclado el usuario y con un if lo eliminamos si existe con .remove o mandamos mensaje de que el usuario no existe.

```

//Eliminar Usuario
case 2:

    System.out.print("Introduce el email del usuario que quieres eliminar: ");
    //creamos una variable para guardar el email que queremos borrar
    String emailEliminar = teclado.nextLine();

//Comprobamos con .containsKey si el email que queremos eliminar esta guardado y si existe lo borramos con el .remove
if (mapaUsuarios.containsKey(emailEliminar)) {
    mapaUsuarios.remove(emailEliminar);
    System.out.println("Usuario eliminado correctamente.");
}

//Si no existe ponemos un mensaje para aclararlo
else {
    System.out.println("El usuario no existe.");
}
break:

```

Case 3:

Nuestro case 3 es **crear evento**, es similar a crear usuario, pedimos por teclado que se asigne valor a los parámetros del evento.

```
//Añadir Evento
case 3:

    //Pasamos la fecha al formato AAAA-MM-DD

    System.out.print("Formato de Fecha (AAAA-MM-DD): ");
    System.out.println("Introduce la Fecha");
    String fechaTexto = teclado.nextLine();
    //Guardamos la fecha introducida
    LocalDate fecha = LocalDate.parse(fechaTexto);
    System.out.print("Introduce el Título: ");
    String titulo = teclado.nextLine();

    System.out.print("Introduce la Ubicación: ");
    String ubicacion = teclado.nextLine();
    System.out.print("Introduce la Descripción: ");
    String descripcion = teclado.nextLine();
```

Guardamos el evento con sus parámetros y su id generado con el contador

```
//Creamos el evento con los datos introducidos y lo guardamos en el HashMap
Evento nuevoEvento = new Evento(contadorIdsEventos, fecha, titulo, ubicacion, descripcion);
mapaEventos.put(contadorIdsEventos, nuevoEvento);
System.out.println("Evento creado con ID: " + contadorIdsEventos);
contadorIdsEventos++;
break;
```

Case 4:

Este case es eliminar un evento. Para hacerlo más sencillo para el usuario que está usando el programa le mostramos con un bucle for y .values la lista de los ids de los eventos para que el usuario sepa cual quiere eliminar a la hora de ingresarlo. Igual que al eliminar el usuario debemos identificar que este evento ya existe, si existe con .remove lo eliminamos y si no existe

mandamos un mensaje que nos informe de que el ID del evento que queremos eliminar no está registrado.

```
//Eliminar Evento
case 4:
    //Primero vamos a mostrar con un bucle for los IDs para que el usuario los vea de manera bonita
    for (Evento IDs : mapaEventos.values()) {
        System.out.println(IDs);}

    // Pedimos el ID del evento que desea eliminar
    System.out.print("\n Introduce el ID del evento que deseas eliminar ");
    int idEliminar = teclado.nextInt();
    teclado.nextLine(); // Limpieza de buffer

    // Verificamos si el evento existe y si existe lo eliminamos
    if (mapaEventos.containsKey(idEliminar)) {
        mapaEventos.remove(idEliminar);
        System.out.println("Evento eliminado correctamente.");
    } else {
        // Si el evento no existe mostramos un mensaje que nos informe
        System.out.println("El evento no existe.");
    }
    break;
```

Case 5:

Vamos a asignar una galería a un evento, como antes vamos a mostrar la lista de los IDs de los eventos al usuario y comprobamos que el que ha introducido exista con containsKey dentro del "If". En el siguiente orden creamos la galería, la añadimos al arraylist del evento y mostramos si se ha realizado con éxito:

```
//Comprobamos si existe el evento al que queremos asignarle la nueva galeria
if (!mapaEventos.containsKey(idEvento)) {
    System.out.println("ID de evento incorrecto.");
} else {
    System.out.print("Introduce el Título de la galería: ");
    String tituloGal = teclado.nextLine();

    //Creamos el objeto Galeria con el contador el título y el ID
    Galeria nuevaGaleria = new Galeria(contadorIdsGalerias, tituloGal, idEvento);

    //Añadimos a la colección de galerías del evento específico usando el Id de la galeria
    //Obtenemos la lista del arrayList de la galeria y añadimos el nuevo objeto creado.
    mapaEventos.get(idEvento).getGalerias().add(nuevaGaleria);

    //Mostramos que se haya creado correctamente y el ID
    System.out.println("Galería creada correctamente, el ID asignado es: " + contadorIdsGalerias);
    contadorIdsGalerias++;
}
break;
```


Case 6

Este case es la opción de eliminar una galería de un evento, para ello mostramos de nuevo el ID de los eventos, pedimos que introduzca uno, se comprueba si existe dicho evento o no, luego mostramos las galerías que tiene dicho evento y pedimos al usuario que introduzca la que quiere eliminar. Si coincide la galería que hemos puesto con una que tenga el evento al recorrer el bucle, la guarda en una nueva variable y posteriormente la elimina con .remove.

```
// Mostramos la lista de galerias del evento
for (Galeria g : eventoSeleccionado.getGalerias()) {
    System.out.println(g); // Usa el toString() de Galeria si lo tenemos creado
}

System.out.print("\nIntroduce el ID de la galería que quieres eliminar: ");
int idGalEliminar = teclado.nextInt();
teclado.nextLine();

/*Usamos una nueva variable inicializada estando vacía, si el ID de g al recorrer
la lista es el mismo que hemos escrito para eliminar lo guarda en la variable galeriaEncontrada*/
Galeria galeriaEncontrada = null;
for (Galeria g : eventoSeleccionado.getGalerias()) {
    if (g.getId() == idGalEliminar) {
        galeriaEncontrada = g;
        break; // Cuando coincide salimos del bucle
    }
}

//Una vez encontrada obtenemos la lista y la borramos
if (galeriaEncontrada != null) {
    eventoSeleccionado.getGalerias().remove(galeriaEncontrada);
    System.out.println("Galeria eliminada correctamente.");
} else {
    System.out.println("No se ha encontrado ninguna galería con ese ID en este evento.");
}

}
```

Case 7:

En el case 7 queremos asignar un evento como favorito de un usuario, para ello pedimos el Id del evento y el correo del usuario. Una vez los tengamos recorreremos el hashmap y si existen guardamos en la lista Favoritos el correo y el id del evento.

```

//Añadir Favorito
case 7:

    System.out.println("Vamos a añadir un evento a favoritos");

    //Mostramos el ID y el título de los eventos al usuario para que escoja uno
    System.out.println("Eventos disponibles:");
    for (Evento eventos : mapaEventos.values()) {
        System.out.println("ID: " + eventos.getId() + " titulo " + eventos.getTitulo());
    }

    //Pedimos el ID del evento y luego el Email del usuario, que son las claves
    System.out.print("\nIntroduce el ID del evento: ");
    idEvento = teclado.nextInt();
    teclado.nextLine(); // Limpiar buffer

    System.out.print("\nIntroduce el Email del usuario: ");
    String emailFav = teclado.nextLine();

    // Verificamos si existen el evento y el usuario con .containsKey()
    if (mapaEventos.containsKey(idEvento) && mapaUsuarios.containsKey(emailFav)) {

        //Si existen creamos el objeto Favorito con los datos introducidos de Usuario(email) y Evento(ID)
        Favorito nuevoFav = new Favorito(emailFav, idEvento);

        //Lo guardamos en la lista de favoritos
        listaFavoritos.add(nuevoFav);

        System.out.println("Favorito creado correctamente");
    } else {
        // Si nos da error
        System.out.println("El ID de evento o el Email de usuario son incorrectos");
    }
}

```

Case 8

Eliminar un evento favorito de un usuario. Mostramos los Favoritos y pedimos por teclado el correo del usuario y el ID del evento que son las claves del HashMap. Con un bucle buscamos en la lista de favoritos los datos que hemos introducido y lo guardamos en una nueva variable para eliminarlo posteriormente.

```

//Bucle para buscar el favorito
Favorito favEliminar = null;
for (Favorito f : listaFavoritos) {
    // Verificamos si coinciden ambos datos y si coinciden lo guardamos en favEliminar
    if (f.getCorreoUsuario().equals(emailBorrar) && f.getIdEvento() == idEvento) {
        favEliminar = f;
        break;
    }
}

//Usamos un if para eliminar el favorito si lo ha guardado y un mensaje de error por si da problemas
if (favEliminar != null) {
    listaFavoritos.remove(favEliminar);
    System.out.println("Favorito eliminado correctamente");
} else {
    System.out.println("El favorito no existe");
}
break;

```

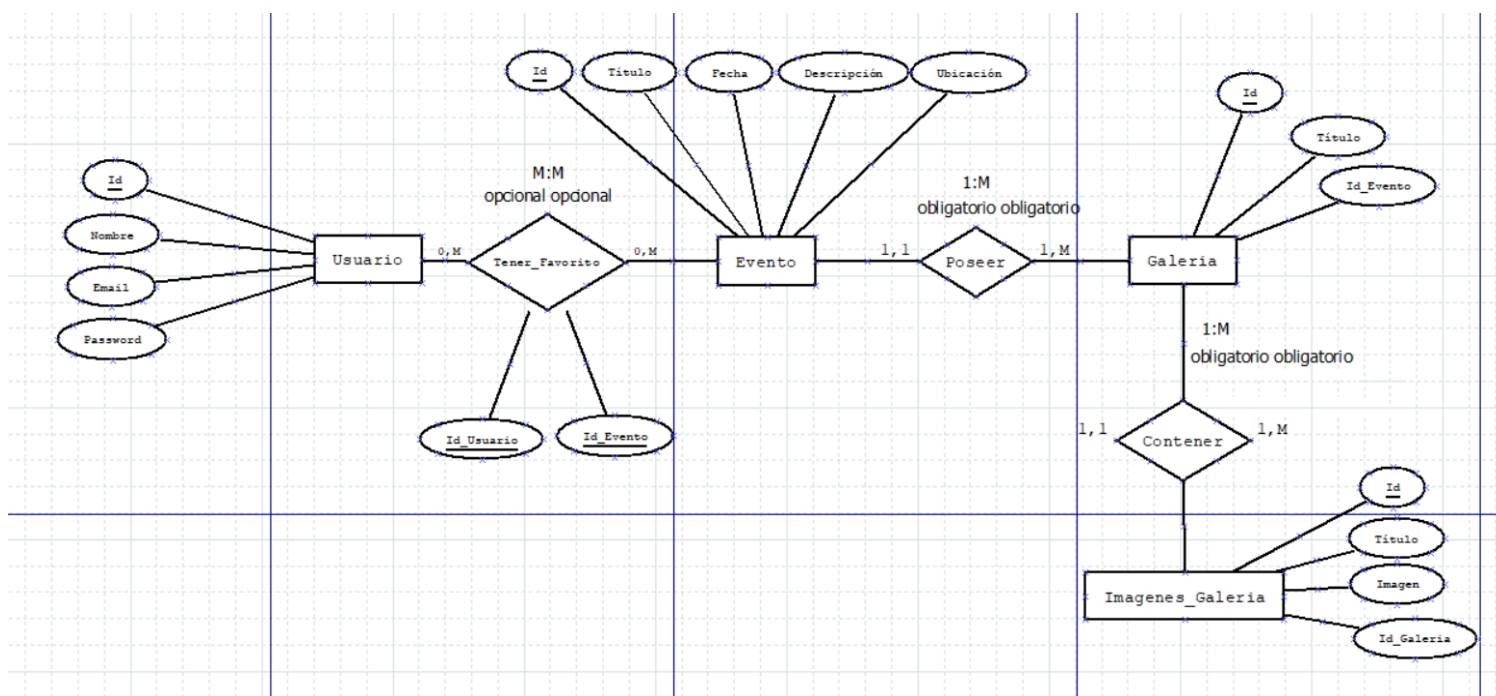
Case 9

El case 9 es para salir del menú. Con esto finalizamos la parte de programación.

- BLOQUE BASE DE DATOS**

El trabajo que debemos desarrollar en este apartado es diseñar una base de datos con un modelo Entidad-Relación en el programa “Dia” y el uso del programa MySQL workbench para la creación e inserción de datos en un archivo sql.

En el diagrama que he creado en Dia.exe se aprecian las tablas con sus claves primarias, foráneas, relaciones y cardinalidades.



Usamos MySQL WorkBench para crear las tablas mediante código y nos queda de la siguiente manera visto desde Visual Studio Code.

Empleando una conexión debemos crear la base de datos con “CREATE DATABASE IF NOT EXIST nombre de la base; Use Happiness”

Luego empezamos con la creación de las tablas Usuarios, Eventos, Galerías, Imágenes de galerías y Favoritos. Lo hacemos con Create table y luego introducimos los atributos y que tipo de atributos registramos. Indicamos las Primary keys y las foráneas con CONSTRAINT.

```

CREATE DATABASE IF NOT EXISTS Happiness;
USE Happiness;

/*Creacion de tablas*/

CREATE TABLE Usuarios (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(60) NOT NULL,
  email VARCHAR(60) UNIQUE NOT NULL,
  password VARCHAR(60) NOT NULL
);

CREATE TABLE Eventos (
  id INT PRIMARY KEY,
  fecha DATE NOT NULL,
  titulo VARCHAR(150) NOT NULL,
  ubicacion VARCHAR(80) NOT NULL,
  descripcion VARCHAR(1000)
);

CREATE TABLE Galerias (
  id INT AUTO_INCREMENT PRIMARY KEY,
  titulo VARCHAR(80) NOT NULL,
  id_Evento INT NOT NULL,
  CONSTRAINT fk_evento_galeria FOREIGN KEY (id_Evento) REFERENCES Eventos(id) ON DELETE CASCADE
);

CREATE TABLE Imagenes_Galerias (
  id INT AUTO_INCREMENT PRIMARY KEY,
  titulo VARCHAR(80) NOT NULL,
  imagen VARCHAR(100) NOT NULL,
  id_Galeria INT NOT NULL,
  CONSTRAINT fk_galeria_imagen FOREIGN KEY (id_Galeria) REFERENCES Galerias(id) ON DELETE CASCADE
);

CREATE TABLE Favoritos (
  id_usuario VARCHAR(60) NOT NULL,
  id_Evento INT NOT NULL,
  PRIMARY KEY (id_usuario, id_Evento),
  CONSTRAINT fk_email_usuario_favorito FOREIGN KEY (correo_usuario) REFERENCES Usuarios(email) ON DELETE CASCADE,
  CONSTRAINT fk_evento_favorito FOREIGN KEY (id_Evento) REFERENCES Eventos(id) ON DELETE CASCADE
);

```

Para la creación de las tablas debemos seguir un orden y empezar por las que no tienen claves foráneas.

El siguiente paso es insertar datos en las tablas mediante el **insert into** indicamos el nombre de la tabla continuamos con **values** e insertamos los datos entre parentesis separados por comas.

Finalmente creamos las vistas que nos pide el proyecto con select para escoger las tablas y diferentes sintaxis para mostrar lo requerido.

```
/*VISTAS*/
```

```
CREATE VIEW Vista_Galerias_Pasadas AS  
SELECT Galerias.* FROM Galerias  
JOIN Eventos ON Galerias.id_Evento = Eventos.id  
WHERE Eventos.fecha < '2026-02-28';
```

Todos los eventos favoritos del primer usuario

```
CREATE VIEW Vista_Favoritos_Juanito AS  
SELECT Eventos.* FROM Eventos  
JOIN Favoritos ON Eventos.id = Favoritos.id_Evento  
WHERE Favoritos.correo_usuario = 'primerusuario@gmail.com';
```

Todas las imágenes del evento del 12-01-2026

```
CREATE VIEW Vista_Imagenes_Cine AS  
SELECT Imagenes_Galerias.* FROM Imagenes_Galerias  
JOIN Galerias ON Imagenes_Galerias.id_Galeria = Galerias.id  
WHERE Galerias.id_Evento = 1;
```

Eventos favoritos del usuario 2 posteriores al 28-02-2026

```
CREATE VIEW Vista_Favoritos_Futuros_Elena AS  
SELECT Eventos.* FROM Eventos  
JOIN Favoritos ON Eventos.id = Favoritos.id_Evento  
WHERE Favoritos.correo_usuario = 'elena.mtnez@gmail.com'  
AND Eventos.fecha > '2026-02-28';
```

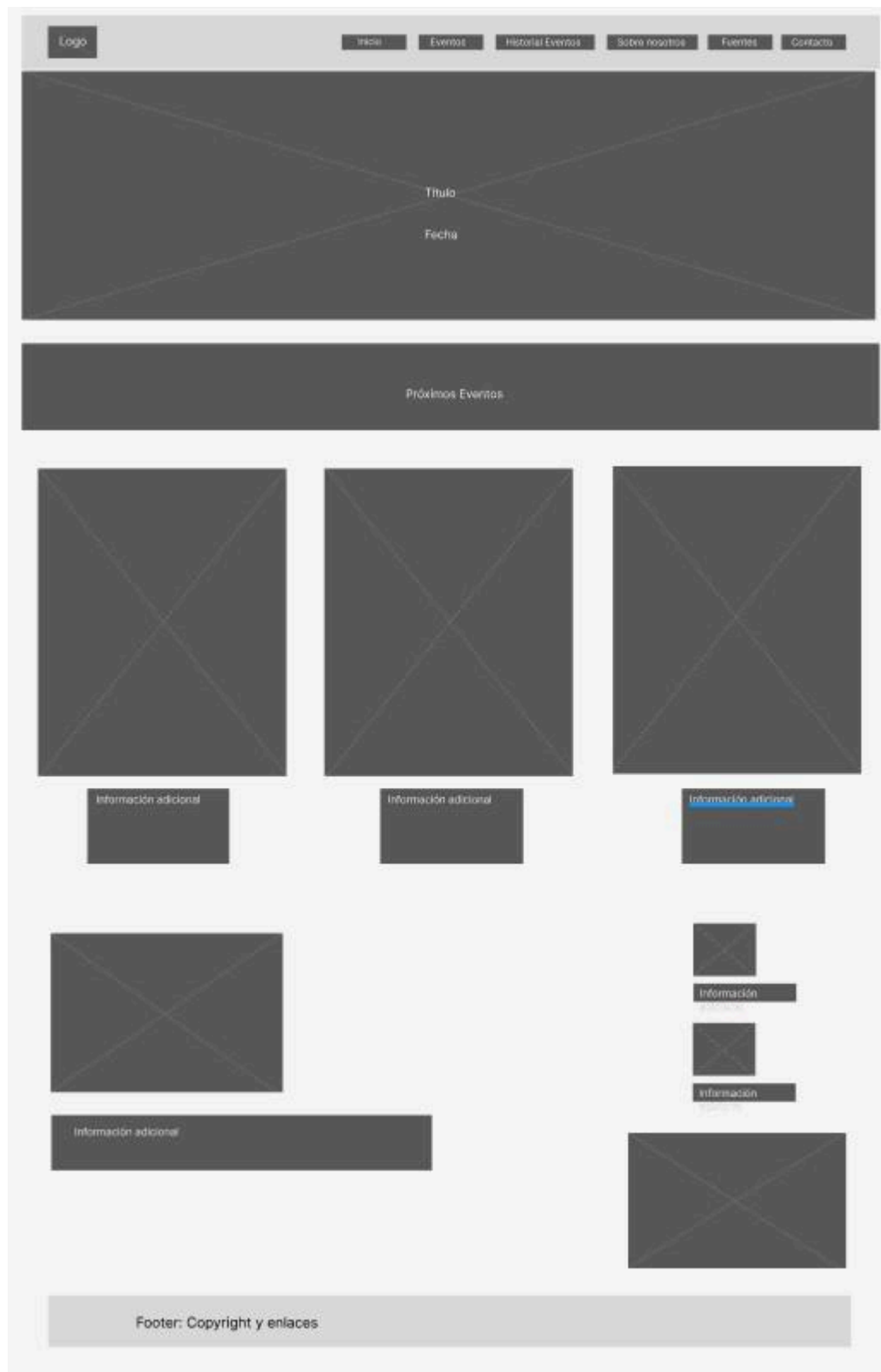
- **BLOQUE DE LENGUAJE DE MARCAS**

Para el apartado de Lenguaje de Marcas se nos pide diseñar una web con html sobre la empresa Happiness&Co. En esta debemos crear un menú de navegación para poder movernos entre las distintas páginas creadas con html, debemos crear los archivos index.html(home), eventos.html(sobre eventos futuros), historial.html(sobre eventos del pasado), fuentes.html, nosotros.html(sobre nosotros) y contacto.html(para que los usuarios se pongan en contacto con nosotros).

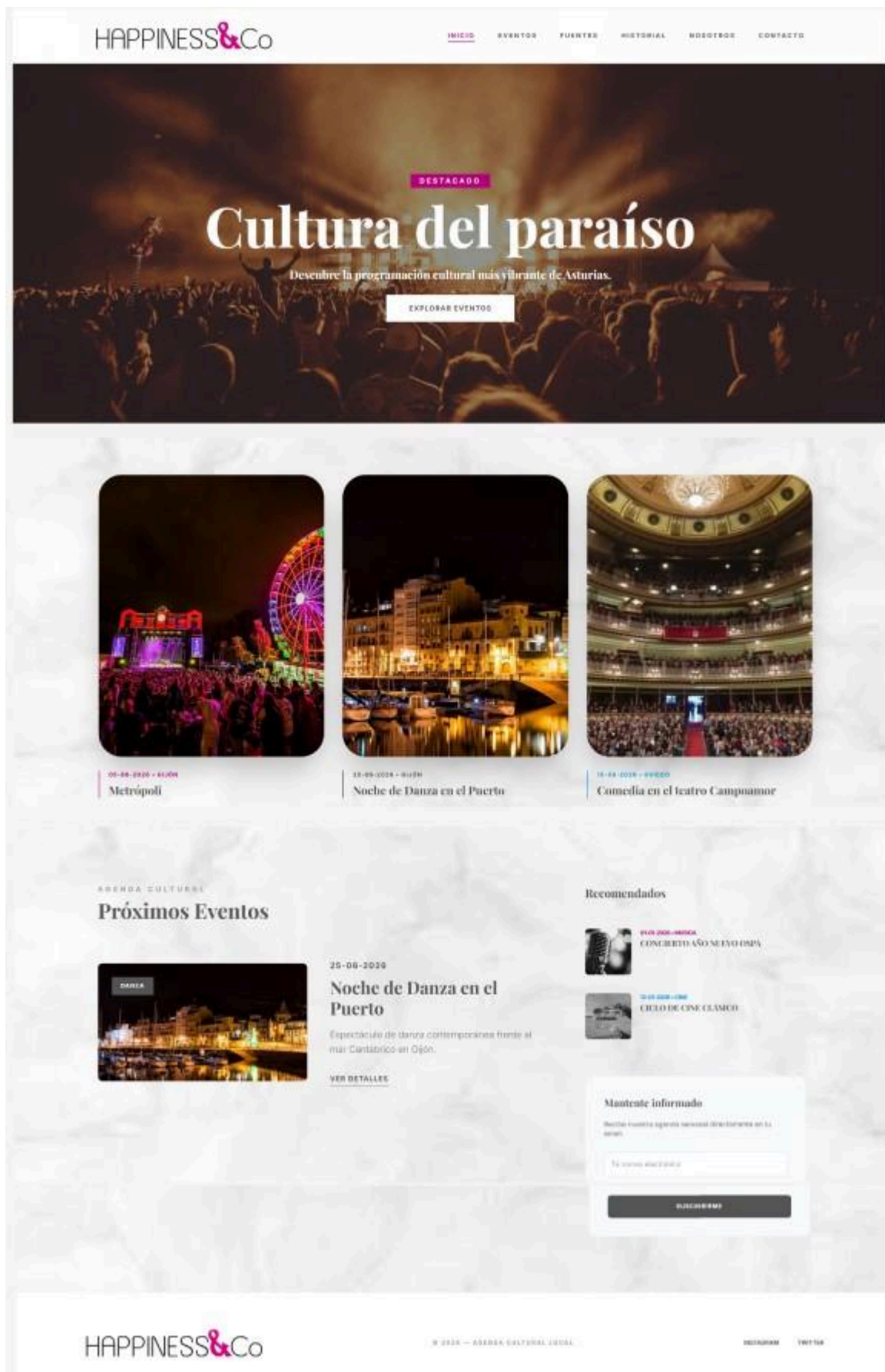
Previamente a realizar las páginas web con Visual Studio Code, empleando css y tailwind, debemos realizar el Wireframe y el Mockup de cómo se van a ver nuestros archivos html en el navegador. Estos diseños nos sirven como plantilla y estructura de cómo se va o queremos que se vea nuestra web.

El wireframe es una plantilla base donde se colocan figuras geométricas en el espacio representando la posición de imágenes, menús, cuadros de texto etc. El Mock-up es una imagen que representa el diseño final de cómo queremos que se vea la web.

Wireframe de index.html



Mock-up de index.html



Estos los hemos creado en el programa Figma.

Una vez creados el wireframe y el mock-up vamos a crear en Visual Studio Code los html el estilo de css y empleando tailwind.

Empleamos tailwind a través de un link en el inicio de nuestro código, con los colores corporativos y link a nuestro css.

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>HAPPINESS & Co - Historial</title>
<script src="https://cdn.tailwindcss.com"></script>
<link rel="stylesheet" href="styles/styles.css">
<script>
  tailwind.config = {
    theme: {
      extend: {
        colors: {
          'h-pink': '#ba007c',
          'h-blue': '#009bdb',
          'h-orange': '#ff7b00',
          'h-grey': '#555555',
        }
      }
    }
  }
</script>
```

El estilo que vamos a emplear en nuestro css es el siguiente, dentro de la carpeta styles:

```
@import url('https://fonts.googleapis.com/css2?family=Playfair+Display:ital,wght@0,700;1,700&family=Inter:wght@300;400;700&display=swap');

:root {
  --h-pink: #ba007c;
  --h-blue: #009bdb;
  --h-orange: #ff7b00;
  --h-grey: #555555;
}

body {
  font-family: 'Inter', sans-serif;
  color: var(--h-grey);
}

.font-logo {
  font-family: 'Playfair Display', serif;
}

.font-ampersand {
  font-family: 'cursive', serif;
  color: var(--h-pink);
}
```


Lo primero que creamos es el menú en la cabeza de la página para movernos por las diferentes web.

```
<body style="background-image: url('assets/img/fondo.jpg'); background-size: cover; background-position: center; background-attachment: fixed;

<header class="sticky top-0 z-50 bg-white/95 backdrop-blur-md border-b border-gray-100">
  <div class="max-w-7xl mx-auto px-6 h-24 flex justify-between items-center">

    <a href="index.html" class="block">
      
    </a>

    <nav class="hidden md:block">
      <ul class="flex gap-10 text-[11px] font-bold uppercase tracking-[0.2em]">
        <li><a href="index.html" class="text-h-pink border-b-2 border-h-pink pb-1">Inicio</a></li>
        <li><a href="eventos.html" class="hover:text-h-pink transition">Eventos</a></li>
        <li><a href="fuentes.html" class="hover:text-h-pink transition">Fuentes</a></li>
        <li><a href="historial.html" class="hover:text-h-pink transition">Historial</a></li>
        <li><a href="nosotros.html" class="hover:text-h-pink transition">Nosotros</a></li>
        <li><a href="contacto.html" class="hover:text-h-pink transition">Contacto</a></li>
      </ul>
    </nav>

  </div>
</header>
```

Debemos incluir una imagen de un evento en grande con el nombre y la información, debajo de esta 3 imágenes de los eventos futuros y unos eventos recomendados en pequeño a la derecha.

Fondo:

```
<section class="relative h-[85vh] flex justify-center overflow-hidden bg-black">

  <div class="relative z-10 text-center text-white px-4 pt-32">
    <span class="inline-block text-xs font-bold tracking-[0.3em] uppercase mb-4 bg-h-pink px-4 py-1">
      Destacado
    </span>
    <h1 class="text-5xl md:text-8xl font-logo robot">Ven a la Metròpli y más eventos!</h1>
  </div>

  <div class="absolute bottom-16 md:bottom-24 z-10 w-full text-center text-white px-4">
    <p class="text-lg md:text-xl font-logo robot max-w-2xl mx-auto mb-8">
      Descubre la programación cultural más vibrante de Asturias.
    </p>
    <a href="#eventos"
      class="bg-white text-h-grey px-10 py-4 font-bold uppercase text-xs tracking-widest hover:bg-h-pink hover:text-white transition">
      Explorar Eventos
    </a>
  </div>
</section>
```

Los 3 eventos en grande:

```
<section class="max-w-7xl mx-auto px-6 py-16 relative z-20">
  <div class="grid md:grid-cols-3 gap-8">

    <div class="group cursor-pointer">
      <div class="rounded-[40px] overflow-hidden shadow-2xl mb-6 aspect-[4/5] bg-gray-200">
        
      </div>
      <div class="px-4 border-l-2 border-h-pink">
        <p class="text-[10px] font-bold robot text-h-pink uppercase tracking-widest">05-06-2026 <span></span> Gijón</p>
        <h3 class="text-xl font-logo robot font-bold mt-1">Metropoli Gijón</h3>
      </div>
    </div>

    <div class="group cursor-pointer">
      <div class="rounded-[40px] overflow-hidden shadow-2xl mb-6 aspect-[4/5] bg-gray-200">
        
      </div>
      <div class="px-4 border-l-2 border-h-grey">
        <p class="text-[10px] font-bold text-h-grey uppercase tracking-widest">25-06-2026 <span></span> Gijón</p>
        <h3 class="text-xl font-logo robot font-bold mt-1">Noche de Danza en el Puerto</h3>
      </div>
    </div>

    <div class="group cursor-pointer">
      <div class="rounded-[40px] overflow-hidden shadow-2xl mb-6 aspect-[4/5] bg-gray-200">
        
      </div>
      <div class="px-4 border-l-2 border-h-blue">
        <p class="text-[10px] font-bold text-h-blue uppercase tracking-widest">15-06-2026 <span></span> Oviedo</p>
        <h3 class="text-xl font-logo robot font-bold mt-1">Comedia en el teatro Campoamor</h3>
      </div>
    </div>
  </div>
</section>
```

Próximos eventos;

```
<main id="eventos" class="max-w-7xl mx-auto px-6 py-24">
  <div class="flex flex-col lg:flex-row gap-20">

    <div class="lg:w-2/3">
      <div class="mb-16">
        <h2 class="text-xs font-bold tracking-[0.4em] uppercase text-gray-400 mb-2">Agenda Cultural</h2>
        <h3 class="text-4xl font-logo robot">Próximos Eventos</h3>
      </div>

      <div class="space-y-20">
        <article class="group">
          <div class="grid md:grid-cols-2 gap-10 items-center">
            <div class="overflow-hidden relative rounded-lg bg-gray-100">
              
              <div
                class="absolute top-6 left-6 bg-h-grey text-white text-[10px] font-bold px-4 py-2 uppercase tracking-widest">
                Danza</div>
            </div>
            <div>
              <span class="text-h-grey font-bold text-sm tracking-widest">25-06-2026</span>
              <h4 class="text-3xl font-logo robot mt-2 mb-4">Noche de Danza en el Puerto</h4>
              <p class="text-gray-500 font-light leading-relaxed mb-6">Espectáculo de danza contemporánea frente al mar Cantabrico en Gijón.</p>
              <a href="#"
                class="inline-block border-b border-h-grey pb-1 text-xs font-bold uppercase tracking-widest hover:text-h-pink transition">Ver detalles</a>
            </div>
          </div>
        </article>
      </div>
    </div>
  </div>
```

Recomendados:

```
<aside class="lg:w-1/3 space-y-16">
  <div>
    <h3 class="text-xl font-logo robot mb-8 border-b border-gray-100 pb-4">Recomendados</h3>
    <div class="space-y-8">
      <div class="flex gap-4 group cursor-pointer">
        <div class="w-20 h-20 bg-gray-200 overflow-hidden shrink-0 rounded-md">
          
        </div>
        <div>
          <p class="text-[10px] font-bold text-h-pink uppercase tracking-tighter">01-01-2026 </p>
          <h5
            class="text-sm font-logo robot leading-snug group-hover:text-h-pink transition uppercase">
            Concierto Año Nuevo OSPA</h5>
        </div>
      </div>
      <div class="flex gap-4 group cursor-pointer">
        <div class="w-20 h-20 bg-gray-200 overflow-hidden shrink-0 rounded-md">
          
        </div>
        <div>
          <p class="text-[10px] font-bold text-h-blue uppercase tracking-tighter">12-01-2026 </p>
          <h5
            class="text-sm font-logo robot leading-snug group-hover:text-h-blue transition uppercase">
            Ciclo de Cine Clásico</h5>
        </div>
      </div>
    </div>
  </div>
</aside>
```

Footer:

```
<footer class="bg-white border-t border-gray-100 py-16 px-6">
  <div class="max-w-7xl mx-auto flex flex-col md:flex-row justify-between items-center gap-8">
    <a href="index.html" class="block">
      
    </a>
    <div class="text-[10px] font-bold uppercase tracking-[0.2em] text-gray-400">
      2026 Agenda Cultural Local
    </div>
    <div class="flex gap-8">
      <a href="#" class="text-h-grey hover:text-h-pink transition uppercase text-[10px] font-bold">Instagram</a>
      <a href="#" class="text-h-grey hover:text-h-pink transition uppercase text-[10px] font-bold">Twitter</a>
    </div>
  </div>
</footer>
```

Desarrollamos el resto de los archivos html de la misma manera usando los links correctos a las imágenes que tenemos guardadas en /assets/img/.

- **BLOQUE RSS**

En este apartado del proyecto debemos crear un archivo xml donde nosotros creamos las etiquetas diferenciándolo de html para que no solo sea una página web visual sino también sea interpretable como fuente de datos. Debe tener los eventos futuros.

```
<?xml version="1.0" encoding="UTF-8" ?>
<rss version="2.0" xmlns:atom="http://www.w3.org/2005/Atom">
  <channel>
    <title>HAPPINESS & Co - Agenda Cultural</title>
    <link>../Lenguaje de Marcas/eventos.html</link>
    <description>Recopilación de eventos culturales: música, teatro y exposiciones[cite: 5, 6].</description>
    <language>es</language>
    <lastBuildDate>Thu, 16 Apr 2026 13:00:00 +0200</lastBuildDate>

    <item>
      <title>Metropoli Gijón Preview</title>
      <link>../Lenguaje de Marcas/evento-metropoli.html</link>
      <description>Avance del festival de cultura pop y entretenimiento en Gijón.</description>
      <category>Música</category>
      <pubDate>Fri, 05 Jun 2026 10:00:00 +0200</pubDate>
    </item>
  </channel>
</rss>
```

```
    <item>
      <title>Comedia en el Teatro Campoamor</title>
      <link>../Lenguaje de Marcas/evento-teatro.html</link>
      <description>Obra de teatro humorística con actores locales en Oviedo.</description>
      <category>Teatro</category>
      <pubDate>Mon, 15 Jun 2026 19:00:00 +0200</pubDate>
    </item>

    <item>
      <title>Noche de Danza en el Puerto</title>
      <link>../Lenguaje de Marcas/evento-danza.html</link>
      <description>Espectáculo de danza contemporánea frente al mar Cantábrico en Gijón.</description>
      <category>Danza</category>
      <pubDate>Thu, 25 Jun 2026 20:00:00 +0200</pubDate>
    </item>
  </channel>
</rss>
```

Programas utilizados:

- Visual Studio Code
- NetbeansIDE
- Figma
- Dia
- MySQL WorkBench

Fuentes:

- Imágenes:

<https://images.unsplash.com/photo-1470229722913-7c0e2dbbafd3>

https://upload.wikimedia.org/wikipedia/commons/f/f1/METROPOLIGIJON_2.jpg

<https://www.motosdeaguaqijon.com/wp-content/uploads/2023/12/Donde-salir-de-fiesta-en-Gijon.jpg>

https://sala11blog.wordpress.com/wp-content/uploads/2017/09/teatro-camp-oamor_xoptimizadax-ksob-1352x900abc.jpg

<https://casacamila.com/wp-content/uploads/2011/05/MUJA-Tyrannosaurus.jpg>

<https://www.valeycentrocultural.org/wp-content/uploads/2024/07/Orquesta.jpg>

- Información y fuentes:

<https://docs.oracle.com/javase/8/docs/api/java/util/HashMap.html>

https://www.w3schools.com/java/java_arraylist.asp

<https://dev.mysql.com/doc/workbench/en/https://dev.mysql.com/doc/workbench/en/>

<https://www.baeldung.com/java-collections>

<https://developer.mozilla.org/es/docs/Glossary/Semantics>

<https://fonts.google.com/knowledge>

<https://css-tricks.com/snippets/css/a-guide-to-flexbox/>

<https://www.refactoringui.com/>

<https://heroicons.com/>

<https://responsively.app/>